

RELIABLE MQTT FOR PYTHON

reliomq

คู่มือเริ่มต้นใช้งาน

ส่งข้อมูลจาก Sensor ผ่าน MQTT อย่างเข้าใจง่าย พร้อม retry, Outbox และ DeliveryAck

ปรับปรุงสำหรับ reliomq 0.6.2

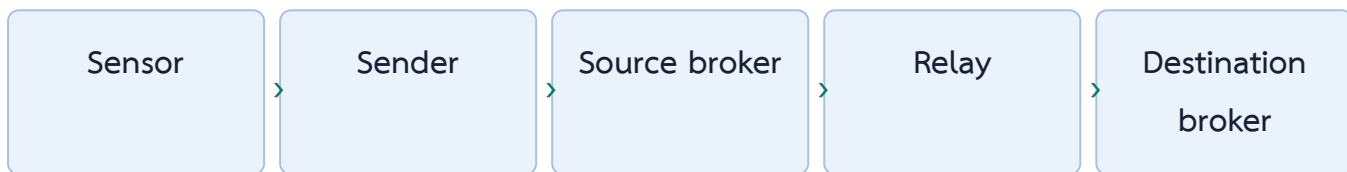
เหมาะสำหรับงาน IoT, Raspberry Pi และ Industrial MQTT

คู่มือนี้ช่วยอะไรคุณ

เมื่อทำตามจบ คุณจะติดตั้ง reliomq, เปิด Relay, ส่งค่าจาก Sensor จำลอง และตรวจได้ว่าข้อมูลถึงปลายทางจริงหรือยัง โดยไม่ต้องรู้โครงสร้างภายในของไลบรารีทั้งหมด

แนวคิดสำคัญที่สุด: MQTT broker ยืนยันได้เพียงว่ารับ publish หนึ่งครั้งแล้ว แต่ **DeliveryAck** ของ reliomq ใช้ยืนยันว่าข้อมูลผ่าน Relay ไปถึง broker ปลายทางแล้ว จึงค่อยนำข้อความออกจากคิวที่รอส่ง

ภาพรวมการไหลของข้อมูล



Relay จะส่ง DeliveryAck ย้อนกลับผ่าน source broker ไปยัง Sender หลัง destination broker ตอบรับข้อมูล

สิ่งที่ต้องมี

- Python 3.11 ขึ้นไป
- MQTT broker เช่น Mosquitto ที่กำลังทำงาน
- สิทธิ์เขียนไฟล์ในตำแหน่ง `outbox_path`
- สำหรับการทดลอง ใช้ broker เดียวกันที่ `localhost:1883` ได้

ลำดับง่าย่าย

- 1 Config
- 2 Object
- 3 connect
- 4 publish
- 5 disconnect

1. รู้จัก 4 ส่วนหลัก

SenderConfig รวมค่า broker, topic, Outbox, timeout และการเชื่อมต่อของฝั่งส่ง	Sender รับข้อมูลจากโปรแกรม เก็บข้อความที่ยังไม่จบงาน ส่งซ้ำ และรอ DeliveryAck	RelayConfig กำหนด source broker, destination broker และ topic ภายในของ reliomq
--	---	--

Relay

อ่าน envelope จาก source broker แล้ว forward ไป application topic ที่ destination broker

Outbox

คิวอาร์แบบแบ่ง segment สำหรับข้อความที่ต้องกู้คืนหลัง restart ตำแหน่งนี้ต้องแยกต่อ Sender และต้องอยู่บน storage ที่เขียนได้

อย่าสับสน 3 topic: **relay_topic** และ **delivery_ack_topic** เป็นช่องทางภายในระหว่าง Sender กับ Relay ส่วน topic ที่ส่งให้ **sender.publish(...)** คือ application topic ที่ปลายทางจะได้รับ

2. ติดตั้ง

```
python -m venv venv

# Linux / Raspberry Pi
source venv/bin/activate

# Windows PowerShell
venv\Scripts\Activate.ps1

pip install reliomq
```

ตรวจว่า broker ตอบสนองก่อนเริ่ม เช่น เปิด subscriber ทดสอบ:

```
mosquitto_sub -h localhost -p 1883 -t "test/#" -v
```

ใช้จริง: ล็อกเวอร์ชันใน requirements เช่น **reliomq==0.6.2** และทดสอบกับ broker, filesystem และรูปแบบการปิดเครื่องของอุปกรณ์จริง

3. เปิด Relay

สร้างไฟล์ **relay.py** แล้วเปิดค้างไว้ Relay ตัวอย่างนี้ใช้ broker เดียวกันทั้งขาเข้าและขาออกเพื่อให้ทดลองได้ทันที

```
import threading
from reliomq import Relay, RelayConfig

config = RelayConfig(
    source_host="localhost",
    destination_host="localhost",
    source_port=1883,
    destination_port=1883,
    source_client_id="demo-relay-source",
    destination_client_id="demo-relay-destination",
    relay_topic="reliomq/messages",
    delivery_ack_topic="reliomq/acks",
    log_level="INFO",
)

relay = Relay(config)
try:
    relay.connect()
    threading.Event().wait()
except KeyboardInterrupt:
    pass
finally:
    relay.disconnect()
```

อ่านโค้ดทีละส่วน

ส่วน	หน้าที่
<code>source_host</code>	broker ที่รับ envelope จาก Sender
<code>destination_host</code>	broker ที่รับ application message จริง
<code>relay_topic</code>	topic ภายในสำหรับ Sender ส่ง envelope เข้า Relay
<code>delivery_ack_topic</code>	topic ภายในสำหรับ Relay ส่ง DeliveryAck กลับ
<code>relay.connect()</code>	เชื่อมต่อ broker ทั้งสองฝั่งและเริ่ม forwarding worker

`relay_topic` และ `delivery_ack_topic` ต้องตรงกับ Sender ทุกตัวที่ใช้ Relay นี้ และ client ID สองฝั่งของ Relay ต้องไม่ซ้ำกัน

4. ส่งค่าจาก Sensor จำลอง

เปิดอีก terminal แล้วสร้าง `sensor_demo.py` ตัวอย่างนี้ส่งอุณหภูมิและความชื้นทุก 2 วินาที โดยใช้ FastMode ซึ่งเป็นค่าเริ่มต้นของ reliomq 0.6.2

```
import random
import time
from reliomq import Sender, SenderConfig

config = SenderConfig(
    host="localhost",
    port=1883,
    client_id="demo-sensor-01",
    outbox_path="./outbox/sensor01",
    relay_topic="reliomq/messages",
    delivery_ack_topic="reliomq/acks",
    log_level="INFO",
)

with Sender(config) as sender:  # default = FastMode()
    while True:
        data = {
            "temperature": round(random.uniform(24, 35), 2),
            "humidity": round(random.uniform(45, 75), 2),
        }
        message_id = sender.publish(
            "factory/demo/sensor01",
            data,
        )
        print("accepted:", message_id, data)
        print("pending:", sender.pending_count())
        time.sleep(2)
```

`publish()` สำเร็จหมายถึงอะไร?

หมายถึง Sender รับข้อความเข้า workflow แล้ว ไม่ได้หมายความว่าปลายทางได้รับแล้ว หากต้องรอผลของข้อความหนึ่งรายการ ให้ใช้ `sender.wait_for_delivery(message_id, timeout=10)`

ตัวอย่างรอผลแบบมี timeout

```
message_id = sender.publish("factory/alarm", {"active": True})

if sender.wait_for_delivery(message_id, timeout=10.0):
    print("destination confirmed")
else:
    print("still pending; background retry continues")
```

เมื่อ timeout คีนค่า `False` ข้อความยังไม่ถูกทิ้ง Sender จะ retry ต่อใน background ตาม `retry_interval`

5. ค่า SenderConfig ที่ควรรู้

ค่า	ค่าเริ่มต้น	ความหมายและคำแนะนำ
host	ต้องระบุ	IP หรือ hostname ของ source MQTT broker
outbox_path	ต้องระบุ	ที่เก็บคิวอาร์ ควรแยก path ต่อ Sender และสำรอง/monitor พื้นที่ว่าง
port	1883	port ของ broker; TLS มักใช้ 8883 แต่ต้องตั้ง TLS เพิ่มตามระบบ
client_id	สร้างอัตโนมัติ	กำหนดเองเมื่ออยากติดตามง่าย และต้องไม่ซ้ำกับ client ที่ online พร้อมกัน
relay_topic	reliomq/messages	ต้องตรงกับ Relay ไม่ใช่ application topic
delivery_ack_topic	reliomq/acks	ต้องตรงกับ Relay และควรจำกัดสิทธิ์ publish/subscribe ให้ถูกฝั่ง
qos	1	QoS ของการส่งภายใน workflow
delivery_ack_timeout	3.0 s	รอ DeliveryAck ต่อ attempt ก่อนเข้าสู่ retry
mqtt_puback_timeout	2.0 s	รอ MQTT QoS 1 PUBACK ต่อ attempt
retry_interval	10.0 s	ระยะพักก่อนลองส่งใหม่เมื่อ attempt ล้มเหลว
keepalive	60 s	MQTT keepalive ของ connection
reconnect_min_delay	1.0 s	ช่วง reconnect ต่ำสุด
reconnect_max_delay	60.0 s	เพดานช่วง reconnect backoff
log_level	None	ใช้ "INFO" สำหรับสถานะทั่วไป หรือ "DEBUG" ตอนวิเคราะห์ปัญหา

เลือก Delivery Mode แบบสั้นที่สุด

Mode	เหมาะกับ	สิ่งที่ต้องยอมรับ
FastMode ค่าเริ่มต้น	telemetry ถี่, ลด disk write	ข้อความใหม่ที่ยังอยู่เฉพาะ RAM อาจหายเมื่อไฟดับทันที
GroupMode	สมดุลความเร็วกับความทนทาน	ช่วงที่ยังไม่ fsync อาจสูญหาย และ ACK ที่ยังไม่ checkpoint อาจ replay
DurableMode	alarm, event สำคัญ, transaction	fsync มากที่สุด จึงช้ากว่าและเขียน storage มากกว่า

```
from reliomq import FastMode, GroupMode, DurableMode, Sender

sender = Sender(config)                # FastMode default
sender = Sender(config, mode=GroupMode()) # balanced
sender = Sender(config, mode=DurableMode()) # strongest durability
```

รายละเอียดทุกพารามิเตอร์อยู่ในเอกสาร [reliomq Delivery Modes Guide](#)

6. ตรวจสอบว่าระบบทำงานถูกต้อง

1. เปิด `relay.py` และดู log ว่า source กับ destination broker connected
2. เปิด subscriber ที่ application topic:
`mosquitto_sub -h localhost -t "factory/demo/sensor01" -v`
3. เปิด `sensor_demo.py` และดู `message_id`
4. ตรวจสอบว่า subscriber ได้ข้อมูล และ `pending_count()` ลดหลัง DeliveryAck
5. หยุด brokerชั่วคราวแล้วเปิดใหม่ เพื่อทดสอบ retry และ recovery

ปัญหาที่พบบ่อย

อาการ	ตรวจสอบอะไรเป็นลำดับแรก
connect ไม่ได้	ตรวจสอบ host, port, firewall, TLS/auth และดูว่า broker ทำงานอยู่
ส่งได้แต่ pending ไม่ลด	Relay ต้องทำงาน และ topic ภายในสองค่าต้องตรงกันทั้ง Sender/Relay
Relay broker เดียวกันแล้ว client ถูกเตะ	<code>source_client_id</code> กับ <code>destination_client_id</code> ต้องคนละชื่อ
Outbox error หรือคิวผิดเครื่อง	อย่าให้ Sender หลาย process ใช้ <code>outbox_path</code> เดียวกัน
FastMode queue เต็ม	ตรวจสอบพื้นที่ disk/ลิ้นชักเขียน, ลดอัตรา publish หรือปรับ hard limit อย่างระมัดระวัง
มีข้อมูลซ้ำหลัง restart	เป็นไปได้ใน at-least-once delivery; consumer ควร deduplicate ด้วย <code>message_id</code>

Checklist ก่อนต่อ Sensor จริง

- เปลี่ยน `localhost` เป็น broker จริงและเปิด TLS/auth ตามนโยบาย
- กำหนด client ID ไม่ซ้ำ และแยก Outbox ต่อ Sender
- วาง Outbox บน storage ที่มีพื้นที่และลิ้นชักเขียนเพียงพอ
- กำหนด application topic ให้สื่อความหมายและจัด ACL ให้ถูกต้อง
- เลือก Mode ตามผลกระทบของข้อมูลหาย/ซ้ำ ไม่ใช่เลือกจากความเร็วยิ่งเดียว
- ทดสอบ broker ล่ม, network ขาด, process kill และ power cycle
- เปิด INFO log ใน production และใช้ DEBUG เฉพาะช่วงวิเคราะห์

สูตรจำ: Config -> Object -> connect -> publish -> DeliveryAck -> disconnect

หากต้องการ workflow แบบครบถ้วน ต้องมี Relay ทำงาน และ consumer ต้องรองรับข้อมูลซ้ำตามหลัก at-least-once

เอกสารนี้อ้างอิง reliomq 0.6.2 จาก branch main รายละเอียด implementation อาจเปลี่ยนในเวอร์ชันถัดไป